

EGA - Cuestionario – Técnicas Digitales 3

Gallo Acosta Alejandro Martín
e-mail: Alejandro.m.gallo@hotmail.com

1. CUESTIONARIO

Las siguientes preguntas deben contestarse y entregarse en un informe aparte.

1. Sistemas Operativos – Generalidades

- ¿Quién es el encargado de administrar el tiempo de la CPU y cuál es su función principal dentro del sistema?
- Explique a qué se denomina “Contexto de Ejecución”. ¿Cuáles son las variables intervinientes?
- ¿A qué se denomina Sistema Operativo de Tiempo Real? ¿Por qué usar un Sistema Operativo de Tiempo Real?

2. Kernel de FreeRTOS

- Explique y desarrolle las características de una “Tarea” en FreeRTOS. Proponga un ejemplo de una tarea manteniendo la sintaxis correcta.
- Explique y desarrolle los “Estados de una Tarea” y sus transiciones. Indique las funciones que permiten las transiciones.
- ¿A qué nos referimos con el concepto de “tareas de procesamiento continuo”? ¿Y a qué nos referimos con el concepto de “tareas manejadas por eventos”?
- ¿Cuáles son los tipos de eventos por los que una tarea puede estar esperando al entrar en el estado bloqueado? Explique.
- ¿Cuál es la función de la *Idle Task*? ¿Puede el procesador no estar ejecutando ninguna tarea en algún momento? Explique.

- Desarrolle cómo es el almacenamiento de datos en el recurso “Cola” proporcionado por el Kernel.
- Explique utilizando un ejemplo con código el porqué es común que el recurso Cola posea múltiples tareas escritoras y una sola tarea lectora. Explique el proceso de bloqueo por escritura y bloqueo por lectura en el recurso Cola.

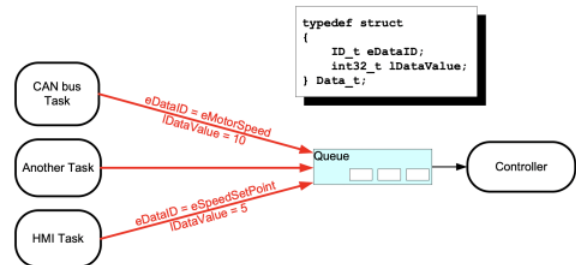


Fig. 1. Recibir datos de múltiples fuentes

- A la hora de enviar datos compuestos: explique cómo se lleva a cabo dicho proceso y por qué no genera conflictos con la definición de la macro del recurso. ¿Cuál sería la implementación si los datos fueran “grandes”, lo que ocasionaría un problema con la memoria RAM?
- Explique el funcionamiento del esquema de la figura 1.
- Desarrolle un programa ejemplo que permita comprender el concepto y uso de “Semáforos Mutex”.
- Desarrolle un programa ejemplo que permita comprender el concepto y uso de “Inversión de Prioridad”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.
- Explique el concepto de “Herencia de Prioridad” utilizando diagramas gráficos que permitan comprenderlo.
- Desarrolle un programa ejemplo que permita comprender el concepto de “Punto Muerto”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

1.1 SISTEMAS OPERATIVOS

- El encargado de administrar el tiempo de la CPU es el Scheduler. Su función principal es la de determinar el cambio de estado de las tareas de activa a inactiva.
- El contexto de ejecución es el conjunto de recursos que describe el estado actual de una tarea en el procesador, para que el scheduler la reanude desde el punto que se interrumpió.

Las variables intervinientes son:

- Los **registros internos del microprocesador**.
- Pila (stack)** de la tarea.
- Variables internas de FreeRTOS**

- Un sistema operativo en tiempo real es un tipo de software que se encarga de gestionar los recursos de hardware y software de una computadora, así como de garantizar que las tareas se ejecuten dentro de un lapso de tiempo determinado.

Las ventajas son:

- Determinismo en el programa, teniendo el control de los tiempos de ejecución y respuesta.
- Simula trabajar en "multitarea".

1.2 KERNEL DEL FREERTOS

- Una tarea es una función que se ejecuta de manera independiente bajo el control del scheduler. Cada tarea tiene su propio **stack**, **prioridad y estado**, y puede **bloquearse, suspenderse o ejecutarse** según la planificación. Permite ejecutar múltiples procesos "en paralelo" compartiendo la CPU.

Las tareas correrán de forma permanente en un lazo infinito, y nunca saldrá. Las tareas de FreeRTOS no deben poder retornar de su función de implementación.

Características:

- Función con prototipo: **`void vTarea(void *pvParameters);`**
- Propia pila y variables locales.
- Priorización configurable.
- Control mediante funciones del kernel (`vTaskDelay`, `vTaskSuspend`, etc.).

Ejemplo:

```
void vTask1(void *pvParameters) {
    for(;;) {
        vPrintString("Task 1 is running\r\n");
        vTaskDelay(250 / portTICK_RATE_MS);
    }
}
```

- Los estados de una tarea en FreeRTOS son:

- Running:** la tarea está siendo ejecutada por el procesador.
- Ready:** está disponible para ejecutarse pero no ha sido seleccionada por el scheduler.
- Blocked:** espera que ocurra un evento (por ejemplo, llegada de datos a una cola).
- Suspended:** ha sido suspendida explícitamente y no puede ser ejecutada hasta que se reanude.

El paso de un estado a otro se da por funciones como `vTaskDelay()`, `vTaskSuspend()`, `vTaskResume()`, `vTaskStartScheduler()`.

- Las tareas de procesamiento continuo son aquellas que siempre tienen trabajo que hacer y no se bloquean. Por ejemplo, una tarea que monitorea constantemente un sensor.

Las tareas manejadas por eventos se ejecutan solo cuando ocurre un evento, como la llegada de datos a una cola o la activación de una interrupción.

- Una tarea puede estar bloqueada esperando:

- Eventos temporales:** como un retardo (`vTaskDelay()` o `vTaskDelayUntil()`).
- Eventos de sincronización:** como la llegada de datos a una cola, la disponibilidad de un semáforo, etc.

- La Idle Task es una tarea especial que se ejecuta cuando no hay ninguna otra tarea lista para ejecutarse. Tiene la prioridad más baja y puede utilizarse para medir el tiempo de CPU libre o para colocar el procesador en modo de bajo consumo. El procesador siempre ejecuta alguna tarea, por lo tanto, si no hay tareas activas, se ejecuta la tarea ociosa.

1.3 ALMACENAMIENTO DE DATOS EN EL RECURSO "COLA"

Las colas en FreeRTOS permiten la comunicación entre tareas mediante el almacenamiento de datos en forma FIFO (First In, First Out). Cada cola tiene una longitud máxima y un tamaño fijo por elemento, definidos al momento de su creación. Los datos se copian byte a byte al escribir (`xQueueSendToBack()`) y al leer

(xQueueReceive()), lo que garantiza la integridad de la información.

1.4

Es común que una cola posea múltiples tareas escritoras y una sola lectora, porque como al leerse el dato de la cola, éste se borra, las demás tareas lectoras no tendrían registro de ese dato que pasó por la cola.

```
typedef struct{
float valor_temp;
uint32_t valor_hum;
}Datos;
QueueHandle_t cola_datos;
static void ReadTEMP(void *PvParameters);
static void ReadHUM(void *PvParameters);
static void ControlarSalida(void *PvParameters);
int main(void)
{
xTaskCreate(ReadTEMP, "",
configMINIMAL_STACK_SIZE, NULL, 1, NULL);
xTaskCreate(ReadHUM, "",
configMINIMAL_STACK_SIZE, NULL, 1, NULL);
xTaskCreate(ControlarSalida, "",
configMINIMAL_STACK_SIZE, NULL, 1, NULL);
cola_datos = xQueueCreate(2, sizeof(Datos));
vTaskStartScheduler();
while (1)
{
}
static void ReadHUM(void *PvParameters)
{
Datos lectura;
TaskYIELD();
while(1)
{
lectura.valor_hum = Hum();
xQueueSend(cola_datos, &lectura,0);
vTaskDelay(100/portTICK_PERIOD_MS);
}
static void ControlarSalida(void *PvParameters)
{
Datos lectura[2];
vTaskDelay(50/portTICK_PERIOD_MS);
while(1)
{
xQueueReceive(cola_datos, &lectura[0],
portMAX_DELAY);
xQueueReceive(cola_datos, &lectura[1],
portMAX_DELAY);
if(lectura[0].valor_temp > 25 &&
lectura[1].valor_hum > 50)
HAL_GPIO_WritePin(SALIDA_GPIO_Port,
SALIDA_Pin, GPIO_PIN_SET);
else
HAL_GPIO_WritePin(SALIDA_GPIO_Port,
SALIDA_Pin, GPIO_PIN_RESET);
}
}
```

- **Bloqueo por escritura:** ocurre si la cola está llena. La tarea escritora puede bloquearse hasta que haya espacio disponible.
- **Bloqueo por lectura:** ocurre si la cola está vacía. La tarea lectora puede bloquearse hasta que llegue un nuevo dato.

FreeRTOS desbloquea la tarea de mayor prioridad que esté esperando, lo que permite una gestión eficiente de los recursos.

1.5

Para enviar datos compuestos, se utiliza una estructura (struct) que contiene múltiples campos. Esto no genera conflictos con la definición de la cola, ya que se define el tamaño del elemento al crearla. Para datos grandes, se recomienda enviar punteros en lugar de copiar los datos, para ahorrar RAM. Se debe asegurar que el acceso a la memoria sea seguro y que solo una tarea la modifique a la vez. Además, se debe garantizar que la memoria apuntada siga siendo válida durante toda la operación.

1.6 EXPLICACIÓN FIGURA 1.

La Figura 1 representa un esquema típico de comunicación entre múltiples tareas en un sistema embebido basado en FreeRTOS, utilizando una cola compartida como mecanismo de sincronización y transferencia de datos. Este modelo permite que varias tareas emisoras envíen información estructurada a una tarea receptora centralizada, que se encarga de procesar los datos recibidos.

En el esquema se observan tres tareas emisoras: CAN Bus Task, HMI Task y Otra Task, cada una de las cuales genera datos relevantes para el sistema. Estos datos se encapsulan en una estructura definida como:

```
typedef struct
{
ID_t eDataID;
int32_t lDataValue;
} Data_t;
```

Tiene dos campos:

- **eDataID:** un identificador que indica el tipo de dato (por ejemplo, velocidad del motor, punto de ajuste, etc.).
- **IDataValue:** el valor numérico asociado al dato.

Cada tarea envía instancias de esta estructura a una cola compartida, que actúa como buffer FIFO (First In, First Out). Ej:

- La **CAN Bus Task** envía: eDataID = eMotorSpeed, lDataValue = 10.
- La **HMI Task** envía: eDataID = eSpeedSetPoint, lDataValue = 5.

La tarea controladora es la única que lee de la cola. Esta tarea interpreta el contenido de cada estructura

recibida y ejecuta acciones correspondientes según el tipo de dato. Este enfoque permite centralizar la lógica de control, simplificar el diseño del sistema y garantizar la integridad de los datos mediante el uso de mecanismos seguros de FreeRTOS.

Este modelo es altamente escalable, ya que permite agregar nuevas tareas emisoras sin modificar la lógica de la tarea receptora. Además, al utilizar estructuras, se facilita el envío de datos compuestos, y al emplear punteros (en caso de datos grandes), se optimiza el uso de memoria.

1.7 EJEMPLO SEMÁFOROS MUTEX

```
SemaphoreHandle_t xMutex;

void vTask1(void *pvParameters)
{
    const char *pcString = (const char *) pvParameters;

    for (;;)
    {
        xSemaphoreTake(xMutex, portMAX_DELAY);
        printf("%s", pcString);
        fflush(stdout);
        xSemaphoreGive(xMutex);
        vTaskDelay(100 / portTICK_RATE_MS);
    }
}

void vTask2(void *pvParameters)
{
    const char *pcString = (const char *) pvParameters;

    for (;;)
    {
        xSemaphoreTake(xMutex, portMAX_DELAY);
        printf("%s", pcString);
        fflush(stdout);
        xSemaphoreGive(xMutex);
        vTaskDelay(100 / portTICK_RATE_MS);
    }
}

int main(void)
{
    xMutex = xSemaphoreCreateMutex();

    if (xMutex != NULL)
    {
        xTaskCreate(vTask1, "Task1", 1000, "Mensaje de
Tarea 1\n", 1, NULL);
        xTaskCreate(vTask2, "Task2", 1000, "Mensaje de
Tarea 2\n", 1, NULL);
        vTaskStartScheduler();
    }

    for (;;)
}
```

En este ejemplo se utiliza un **mutex** para evitar el acceso simultáneo a la salida estándar por parte de dos tareas. El mutex se crea antes de iniciar el scheduler, y cada tarea lo toma antes de imprimir y lo libera al finalizar.

Esto garantiza que las cadenas se impriman sin superposición ni errores. El uso de xSemaphoreTake() con portMAX_DELAY asegura que una tarea espere indefinidamente hasta que el mutex esté disponible, evitando condiciones de carrera y protegiendo el recurso compartido.

1.8 INVERSIÓN DE PRIORIDAD

Se presentan dos tareas con distinta prioridad. La tarea de menor prioridad (Task1) lee datos desde una cola, mientras que la de mayor prioridad (Task2) escribe en esa misma cola. Ambas tareas deben tomar un mutex para acceder al recurso compartido.

Puede ocurrir que Task2 (prioridad alta) tome el mutex, escriba en la cola y lo libere. Luego, Task1 (prioridad baja) toma el mutex para leer de la cola. Si en ese momento Task2 necesita volver a escribir, queda bloqueada esperando que Task1 libere el mutex. Esto genera una inversión de prioridad, ya que una tarea de alta prioridad queda a la espera de una de menor prioridad.

Ejemplo:

```
xSemaphoreHandle_t Sem_Mutex;
QueueHandle_t Queue;

void Task1(void *pvParameters) {
    uint8_t grados;
    for (;;) {
        xSemaphoreTake(Sem_Mutex, portMAX_DELAY);
        xQueueReceive(Queue, &grados,
portMAX_DELAY);
        printf("Task1 recibió: %d\n", grados);
        fflush(stdout);
        xSemaphoreGive(Sem_Mutex);
    }
}

void Task2(void *pvParameters) {
    uint8_t grados = 25;
    for (;;) {
        xSemaphoreTake(Sem_Mutex, portMAX_DELAY);
        xQueueSend(Queue, &grados, portMAX_DELAY);
        printf("Task2 envió: %d\n", grados);
        fflush(stdout);
        xSemaphoreGive(Sem_Mutex);
    }
}

int main(void) {
    Sem_Mutex = xSemaphoreCreateMutex();
    Queue = xQueueCreate(10, sizeof(uint8_t));
```

■

```

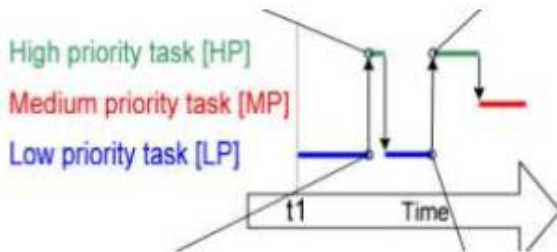
xTaskCreate(Task1, "Task1", 1000, NULL, 1, NULL);
// Prioridad baja
xTaskCreate(Task2, "Task2", 1000, NULL, 2, NULL);
// Prioridad alta

vTaskStartScheduler();
while (1);
}

```

1.9 HERENCIA DE PRIORIDAD

La herencia de prioridad consiste en elevar temporalmente la prioridad de la tarea que posee el mutex, al mismo nivel de prioridad de la tarea de más alta prioridad, que está tratando de obtener el mismo mutex. La tarea de baja prioridad que tiene el mutex "hereda" la prioridad de la tarea que está esperando poder tomar el mutex.



La tarea de baja prioridad toma el mutex y se ejecuta, luego se adelanta la tarea de alta prioridad, pero como la de menor prioridad posee el mutex no puede ejecutarse. En ese momento la tarea de menor prioridad hereda la misma prioridad que la prioridad alta, se ejecuta y luego devuelve el mutex para que se pueda ejecutar la otra tarea de alta prioridad, cuando esta tarea se bloquee entrara la tarea de prioridad media.

1.10 EJEMPLO DE PUNTO MUERTO

Un punto muerto ocurre cuando dos tareas quedan bloqueadas esperando recursos que están en posesión de la otra. En este ejemplo, Task1 toma el mutex A y luego intenta tomar el mutex B. Task2 hace lo contrario: toma el mutex B y luego intenta tomar el mutex A. Si ambas tareas ejecutan en paralelo, pueden quedar bloqueadas mutuamente, sin posibilidad de continuar.

Ejemplo:

```

SemaphoreHandle Sem_Mutex_A;
xSemaphoreHandle Sem_Mutex_B;
QueueHandle_t Queue;

```

```

void Task1(void *pvParameters) {
    uint8_t grados;

```

```

        for (;;) {
            xSemaphoreTake(Sem_Mutex_A,
portMAX_DELAY);
            xQueueReceive(Queue,      &grados,
portMAX_DELAY);
            printf("Task1 recibió: %d\n", grados);
            fflush(stdout);

            // Intenta tomar el segundo mutex
            xSemaphoreTake(Sem_Mutex_B,
portMAX_DELAY);

            // Libera ambos mutex
            xSemaphoreGive(Sem_Mutex_B);
            xSemaphoreGive(Sem_Mutex_A);
        }

void Task2(void *pvParameters) {
    uint8_t grados = 25;
    for (;;) {
        xSemaphoreTake(Sem_Mutex_B,
portMAX_DELAY);
        xQueueSend(Queue,      &grados,
portMAX_DELAY);
        printf("Task2 envió: %d\n", grados);
        fflush(stdout);

        // Intenta tomar el segundo mutex
        xSemaphoreTake(Sem_Mutex_A,
portMAX_DELAY);

        // Libera ambos mutex
        xSemaphoreGive(Sem_Mutex_A);
        xSemaphoreGive(Sem_Mutex_B);
    }

int main(void) {
    Sem_Mutex_A = xSemaphoreCreateMutex();
    Sem_Mutex_B = xSemaphoreCreateMutex();
    Queue = xQueueCreate(10, sizeof(uint8_t));

    xTaskCreate(Task1, "Task1", 1000, NULL, 1,
NULL);
    xTaskCreate(Task2, "Task2", 1000, NULL, 2,
NULL);

    vTaskStartScheduler();
    while (1);
}

```

■